

Terraform Variables, Locals & Outputs

Production Architecture • CI/CD • AWS Secrets Manager • Custom Modules

How values travel through a real Terraform deployment

Customer input → CodeBuild → Terraform variables → module locals → Secrets Manager → AWS resources → outputs → next pipeline stage.

VARIABLES

What the consumer provides

Environment, application name, region, sizing choices and other approved module inputs.

LOCALS

What the Terraform module decides

Naming conventions, mandatory tags, derived values and reusable expressions.

OUTPUTS

What Terraform publishes

Resource IDs, endpoints, ARNs and other values required by automation or the next stage.

The production mental model

```
CUSTOMER / DEVELOPER
|
| approved module inputs
v
CODEPIPELINE / CODEBUILD
|
| TF_VAR_* / -var-file
v
TERRAFORM ROOT MODULE
|
| var.*
v
CUSTOM TERRAFORM MODULE
|
+-----> locals.* -----> naming / tags / sizing
|
+-----> data.* -----> AWS Secrets Manager
| |
| +----> credentials
v
AWS RESOURCES
RDS / ECS / VPC / ALB / ...
```

```
|  
v  
OUTPUTS  
|  
v  
NEXT PIPELINE STAGE / AUTOMATION
```

One rule: expose the minimum inputs needed by the consumer; keep infrastructure decisions inside the module; retrieve secrets at runtime; publish only the outputs that another stage actually needs.

01 — Designing the Terraform Module Interface

A reusable module should feel like a small product: the consumer gets a clear interface and does not need to understand the resources hidden inside the module.

<i>CUSTOMER / APPLICATION TEAM</i>	<i>TERRAFORM MODULE</i>
Provides: • app_name • environment • region • approved sizing/options	Owns: • AWS resource implementation • naming conventions • mandatory tags • derived configuration • internal dependencies

Root configuration — consumer calls the module

```
module "webapp" {  
  source = "../modules/webapp"  
  app_name = "orders"  
  environment = "prod"  
  region = "us-east-1"  
  instance_count = 3  
}
```

Inside the module — variables.tf defines the contract

```
variable "app_name" {  
  description = "Application name supplied by the consumer"  
  type = string  
}  
  
variable "environment" {  
  type = string  
  
  validation {  
    condition = contains(["dev", "stage", "prod"], var.environment)  
    error_message = "Use dev, stage, or prod."  
  }  
}  
  
variable "region" {  
  type = string
```

```
}  
  
variable "instance_count" {  
  type = number  
  default = 1  
}
```

Why this matters: the customer supplies business/environment choices. The module protects everything else. This keeps the module reusable without exposing its implementation.

Functions at the input boundary

```
variable "name_prefix" {  
  type = string  
  default = lower("MyApp-Prod")  
}
```

Functions are useful in supported variable expressions such as defaults and validation. Use validation to reject values the module should never receive.

02 — CodeBuild → Terraform Variables → Locals → Module

The pipeline supplies environment-specific configuration. Terraform remains responsible for turning that configuration into infrastructure decisions.

CodeBuild receives environment values

```
ENVIRONMENT=prod
APP_NAME=orders
AWS_REGION=us-east-1
```

Buildspec maps them into Terraform's input interface

```
phases:
build:
commands:
- export TF_VAR_environment="$ENVIRONMENT"
- export TF_VAR_app_name="$APP_NAME"
- export TF_VAR_region="$AWS_REGION"
- terraform init
- terraform plan -out=tfplan
```

Terraform receives them as **var.environment**, **var.app_name** and **var.region**.

The custom module converts those inputs into internal values

```
locals {
name_prefix = lower("${var.app_name}-${var.environment}")

common_tags = {
Application = var.app_name
Environment = var.environment
ManagedBy = "terraform"
}

instance_type = var.environment == "prod"
? "m6i.large"
: "t3.micro"
}
```

CodeBuild

Terraform

Custom module

AWS resource

ENVIRONMENT=prod	var.environment	local.name_prefix local.common_tags local.instance_type	RDS / ECS / EC2 / ALB
------------------	-----------------	---	-----------------------

Production principle: do not duplicate Terraform naming/tagging/sizing logic in Jenkins or CodeBuild. Let the pipeline provide inputs; let the Terraform module make infrastructure decisions.

Common variable delivery options

Source	Typical use	Production note
TF_VAR_*	CI/CD environment values	Good for non-secret runtime configuration
-var-file	dev/stage/prod configuration	Explicit and easy to review
-var	One-off override	Use sparingly in production
AWS Parameter Store / Secrets Manager	Sensitive/runtime values	Prefer secret-management services for secrets

03 — Secrets Manager → Terraform → AWS Resource

Passwords, API keys and other sensitive values should not travel through Git, committed tfvars files or ordinary pipeline parameters. The runtime identity should retrieve them from the secret store.

AWS Secrets Manager stores the secret

```
Secret name: prod/webapp/database
```

```
{
"username": "appuser",
"password": "*****",
"database": "webapp"
}
```

Terraform reads the existing secret

```
data "aws_secretsmanager_secret_version" "db" {
secret_id = "prod/webapp/database"
}

locals {
db_credentials = jsondecode(
data.aws_secretsmanager_secret_version.db.secret_string
)
}
```

The module/resource consumes the retrieved values

```
resource "aws_db_instance" "postgres" {
identifier = "webapp-prod-db"
engine = "postgres"

username = local.db_credentials.username
password = local.db_credentials.password
db_name = local.db_credentials.database
}
```

CodeBuild	IAM execution role	Secrets Manager	Terraform	RDS
Runs Terraform	GetSecretValue permission	Returns secret	data.* → local.*	Consumes password

IAM should grant only the required secret access. If a customer-managed KMS key protects the secret, the execution identity also needs the appropriate decrypt permission.

State warning: marking a value as sensitive prevents casual display, but it does not make the value magically disappear from Terraform state. Protect remote state with encryption, strict IAM and appropriate locking.

Avoid this pattern

```
terraform apply -var="db_password=SuperSecretPassword123!"
```

Prefer: CodeBuild authenticates to AWS → Terraform reads Secrets Manager → resource receives the value.

04 — Outputs → Production Pipeline Handoff

Outputs are the interface from Terraform back to automation. A production pipeline can capture resource IDs, DNS names and ARNs and use them in later stages.

Terraform publishes selected values

```
output "vpc_id" {
  description = "VPC created by the module"
  value = aws_vpc.main.id
}

output "load_balancer_dns" {
  description = "Application load balancer DNS"
  value = aws_lb.app.dns_name
}
```

After apply, capture the outputs as JSON

```
terraform apply tfplan
terraform output -json > terraform-outputs.json
```

PLAN	APPROVAL	APPLY	CAPTURE	NEXT STAGE
terraform plan -out=tfplan	Human / policy approval	terraform apply tfplan	terraform output -json	Deploy app Smoke tests Configuration

Example output artifact

```
{
  "vpc_id": {
    "value": "vpc-0123456789abcdef0",
    "type": "string"
  },
  "load_balancer_dns": {
    "value": "webapp-prod-123.us-east-1.elb.amazonaws.com",
    "type": "string"
  }
}
```

The next stage can consume these values without recreating or rediscovering AWS resources.

Sensitive outputs

```
output "secret_value" {  
  value = local.db_credentials.password  
  sensitive = true  
}
```

Best practice: do not output a secret unless another component genuinely needs it. Prefer outputting identifiers, endpoints and non-sensitive connection metadata.

05 — Production Reference Architecture

Put the pieces together: the pipeline supplies only approved configuration, Terraform modules contain the infrastructure logic, AWS supplies secrets at runtime, and outputs become controlled handoffs.

```
CUSTOMER / DEV TEAM
|
| app_name / environment / region
v
+-----+
| CODEPIPELINE |
| CODEBUILD   |
+-----+
|
| TF_VAR_* / -var-file
v
+-----+
| TERRAFORM ROOT |
| MODULE         |
+-----+
|
| var.*
v
+-----+
| CUSTOM MODULE |
| |
| locals.* |
| data.* |
+-----+
|
+-----+-----+
| |
v v
+-----+ +-----+
| AWS SECRETS | | AWS RESOURCES |
| MANAGER | | RDS / ECS / |
| credentials | | VPC / ALB... |
+-----+-----+
| |
+-----+
|
v
+-----+
| OUTPUTS |
| IDs / DNS / |
| ARNs / status |
```

```
+-----+-----+
|
|
v
NEXT PIPELINE STAGE
Deploy / Configure / Test
```

Production checklist

Design area	Recommended practice
Module interface	Expose only the inputs the consumer genuinely needs.
Environment values	Use CodeBuild/CI variables or explicit environment tfvars.
Internal logic	Use locals for naming, tags, derived sizing and reusable expressions.
Secrets	Retrieve from AWS Secrets Manager at runtime using an IAM execution role.
State	Use protected remote state; treat state as sensitive.
Outputs	Publish only values needed by humans, automation or downstream stages.
Pipeline	Plan → approval → apply → capture outputs → deploy/test.

The one-line model: Variables define what the consumer may provide → locals define what Terraform derives → data sources read what already exists → resources create/manage infrastructure → outputs publish what the next stage needs.